

DOKUMENTACJA Z PRZEBIEGU TESTÓW AUTOMATYCZNYCH

Student Planner (Kivy/SQLite)

Desktopowa aplikacja produktywnościowa
Inżynieria Oprogramowania
8 czerwca 2026

I. Metryka Środowiska Testowego

Testy zostały przeprowadzone w środowisku heterogenicznym, co pozwoliło na weryfikację stabilności aplikacji na różnych platformach i wersjach interpretera.

Środowisko A (główne środowisko): macOS 15.7 (Apple Silicon ARM64), Python 3.9.6, Kivy 2.3.1

Środowisko B (Środowisko CI/Linuksowe): Linux (Ubuntu/Generic x86_64), Python 3.13.12

Framework Testowy: pytest (wersje 8.4.2 – 9.0.3), pytest-html

II. Cel i zakres testów

Głównym celem procesu QA było zapewnienie niezawodności kluczowych modułów systemu przed wydaniem wersji produkcyjnej. Zakres testów integracyjnych i jednostkowych obejmował:

Moduł bazy danych: operacje tworzenia, odczytu, aktualizacji, kaskadowe usuwanie, zarządzanie stanem.

Moduł formularzy (dodawanie wydarzeń): walidacja wejścia (dane tekstowe, daty, godziny, wartości ujemne), generowanie komunikatów o błędach.

Moduł widoków (kalendarz): logika renderowania dni, filtrowanie wydarzeń, zarządzanie stanem widoku.

Moduł postępów (progress tracker): obliczenia matematyczne postępów, blokady wartości ujemnych.

Szczególną uwagę poświęcono testom przypadków granicznych, obejmujących między innymi niepoprawne formaty dat i godzin, wartości ujemne, bardzo duże wartości liczbowe, puste pola formularzy, długie ciągi znaków oraz dane zawierające polskie znaki i symbole Unicode.

III. Historia iteracji i rozwiązywania defektów

Proces testowy przyjął formę iteracyjną. W początkowych fazach zidentyfikowano błędy walidacyjne i konfiguracyjne, które zostały sukcesywnie wyeliminowane w kolejnych fazach.

1. Moduł: Dodawanie Przedmiotu (test_dodaj_przedmiot.py)

Faza początkowa (30 V 2026): Wykonano 43 testy. Wykryto 16 niezgodności.

Opis defektów: Aplikacja niepoprawnie reagowała na skrajne dane wejściowe. Walidator przepuszczał puste pole prowadzącego, wartości ujemne (np.

ujemny czas trwania, ujemne punkty) oraz nieprawidłowe formaty dat (np. "abc", "2027/01/30"). Zamiast zablokować zapis i wyświetlić okno błędu (show_error_popup), system pozwalał na dodanie wadliwych danych do bazy.

Faza finalna (31 V 2026): Wdrożono poprawki w warstwie logiki formularza. Dodano dodatkowy test. Wykonano 44 testy. Wynik: 100% Passed.

2. Moduł: Egzaminy i Kolokwia (test_egzaminy_i_kolokwia.py)

Faza początkowa (29 V 2026): Wykonano 32 testy. Wykryto 4 błędy integracyjne.

Opis defektów: Problemy ujawniły się podczas testowania walidacji niepoprawnych dat (np. „2026-05-32”, „2026-13-10”, „2026-00-15”, „2026-05-00”). Zamiast poprawnie odrzucić błędne dane i wyświetlić komunikat walidacyjny, aplikacja zgłaszała wyjątek frameworka Kivy. Analiza wykazała, że problem był związany zarówno z obsługą przypadków walidacyjnych, jak i z niepełną inicjalizacją komponentów graficznych Kivy w środowisku testowym. W rezultacie część testów nie mogła zostać poprawnie wykonana.

Faza finalna (31 V 2026): Wprowadzono poprawki w logice walidacji dat oraz dostosowano konfigurację środowiska testowego dla frameworka Kivy. Dodatkowo rozszerzono zestaw testów o bardziej rygorystyczną weryfikację formatów dat i godzin (np. „23:60”, „25:00”, „-1:30”, „YYYY-MM-DD”, „2026-5-5”). Łącznie wykonano 44 testy. Wynik końcowy: 100% Passed.

3. Moduł: Kalendarz (test_calendar.py)

Faza weryfikacji (31 V 2026): Wykonano 21 testów na środowisku Linux/Python 3.13. Wykryto 2 błędy środowiska testowego.

Zidentyfikowane w logach anomalie dotyczyły wyłącznie warstwy izolacji testów jednostkowych. Logika biznesowa aplikacji w każdym z tych przypadków działała zgodnie ze specyfikacją. Powyższe korekty pozwoliły na uzyskanie pełnego, pozytywnego wyniku testów (100% Passed).

4. Moduły stabilne (Baza danych i Tracker progresu)

Moduły te wykazały 100% skuteczności od pierwszej iteracji.

Baza Danych (test_database_function.py): 30 testów. Przetestowano limity (CHECK constraints), kaskadowe usuwanie powiązań i logikę zapisywania unikalnych notatek. Wynik: 100% Passed.

Tracker (test_progress.py): 5 testów. Weryfikacja formatowania tekstu i blokady ujemnych punktów przeszła bez zarzutu. Wynik: 100% Passed.

IV. Zestawienie defektów i uwagi techniczne

Większość problemów dotyczyła walidacji danych w formularzach oraz konfiguracji środowiska testowego.

Największą liczbę błędów wykryto w module dodawania przedmiotów. Testy wykazały, że aplikacja akceptowała część nieprawidłowych danych wejściowych, takich jak wartości ujemne, błędne formaty dat czy niektóre niekompletne dane formularza. Po wprowadzeniu dodatkowych mechanizmów walidacji wszystkie wykryte problemy zostały usunięte, a moduł uzyskał pełny wynik pozytywny.

W module egzaminów i kolokwii początkowo zidentyfikowano problemy ujawniające się podczas testowania niepoprawnych dat i godzin. W wybranych przypadkach aplikacja zamiast poprawnie obsłużyć błędne dane zgłaszała wyjątek frameworka Kivy. Analiza wykazała, że problemy były związane zarówno z obsługą przypadków walidacyjnych, jak i konfiguracją środowiska testowego. W kolejnych iteracjach dopracowano mechanizmy walidacji oraz dostosowano konfigurację testów, rozszerzając jednocześnie zestaw przypadków testowych o dodatkowe niepoprawne formaty dat i godzin. Po wprowadzeniu poprawek wszystkie testy modułu zakończyły się wynikiem pozytywnym.

Podczas testowania modułu kalendarza odnotowano dwa nieudane przypadki testowe. Pierwszy wynikał z niepoprawnego przygotowania obiektu testowego odpowiedzialnego za sprawdzanie wywołania metody odświeżania kalendarza. Drugi był związany z brakiem inicjalizacji obiektu aplikacji w środowisku testowym. W obu przypadkach problem dotyczył wyłącznie samych testów, a nie działania aplikacji. Logika modułu kalendarza działała zgodnie z założeniami projektowymi.

Moduły bazy danych oraz śledzenia postępów nie wykazały żadnych błędów podczas całego procesu testowania. Wszystkie przypadki testowe zakończyły się wynikiem pozytywnym już w pierwszej iteracji.

V. Podsumowanie

Przeprowadzony proces testowania pozwolił na wykrycie oraz usunięcie wszystkich istotnych problemów zidentyfikowanych w trakcie rozwoju aplikacji. Weryfikacja obejmowała zarówno testy jednostkowe i integracyjne wykonywane automatycznie przy użyciu frameworka pytest, jak również testy ręczne polegające na sprawdzaniu działania poszczególnych funkcji z perspektywy użytkownika końcowego.

Po wprowadzeniu niezbędnych poprawek wszystkie moduły systemu uzyskały wynik 100% pozytywnie zakończonych testów automatycznych. Dodatkowo przeprowadzone testy ręczne nie wykazały błędów uniemożliwiających korzystanie z aplikacji ani problemów wpływających na poprawność działania jej kluczowych funkcji.

Uzyskane wyniki potwierdzają stabilność, poprawność oraz spójność działania aplikacji Student Planner. Na podstawie przeprowadzonych testów można stwierdzić, że system spełnia założenia projektowe i jest gotowy do użytkowania.

